

Name of the Student	SHAM S.
Reg. No	192325076
GitHub Link	https://github.com/Sham-2005/MLA0403-Deep_learning

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL 1
Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA04

SLOT :

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Duration: 90 minutes

Assessment Tool Weightage: 25%

Marks: 25

Code of Conduct:

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Name of the student:

Criteria	Conceptual Understanding (2.5)	Linkages (2.5)	Organization (2)	Integration of Literature (2)	Clarity & Presentation (1)	Total (10)
Weightage	25 %	25%	20%	20 %	10 %	100 %
Q1						
Q2						

Total						
-------	--	--	--	--	--	--

Signature of the Faculty:

Total Marks :

QUESTIONS: 02

Total Marks: 20

Case Study Question 1: CNN for Automated Medical Image Classification (10 Marks) A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

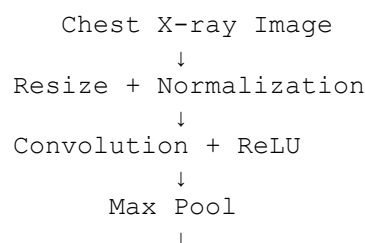
1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.
4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

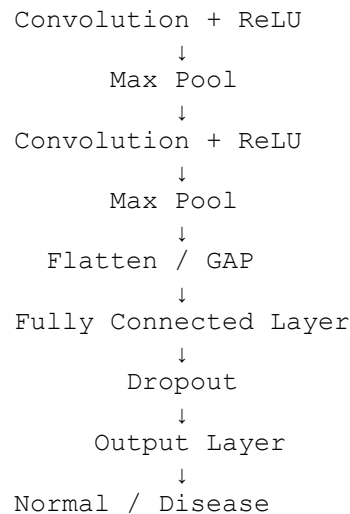
Case Study: CNN for Automated Medical Image Classification

A **Convolutional Neural Network (CNN)** is highly suitable for classifying chest X-ray images because it can automatically learn spatial features such as edges, textures, shapes, and disease-related patterns directly from images.

1. Suitable CNN Architecture

A suitable architecture can be designed as follows:





Example architecture

Layer	Function
Input	$224 \times 224 \times 1$ X-ray image
Conv 3×3 + ReLU	Extract low-level features
Max Pool 2×2	Reduce spatial dimensions
Conv 3×3 + ReLU	Learn textures and patterns
Max Pool 2×2	Downsample feature maps
Conv 3×3 + ReLU	Learn complex abnormal structures
Max Pool 2×2	Further dimensionality reduction
Global Average Pooling	Convert feature maps into compact representation
Dense Layer	Perform high-level classification
Dropout	Reduce overfitting
Output	Normal / Disease

The images should first be **resized to a common resolution** and normalized. Data augmentation can be applied during training to handle variations in brightness, image quality and positioning.

2. Motivation and Role of CNN Components

Why CNNs?

Traditional machine learning requires manually extracting features from X-rays. CNNs perform **automatic feature extraction and representation learning**.

For example:

Early layers → Edges
↓
Middle layers → Textures and shapes
↓
Deep layers → Complex anatomical/abnormal patterns
↓
Output → Disease / Normal

A. Convolutional Layers

A convolutional layer applies small filters to the image to detect important local patterns.

For example, a filter can detect:

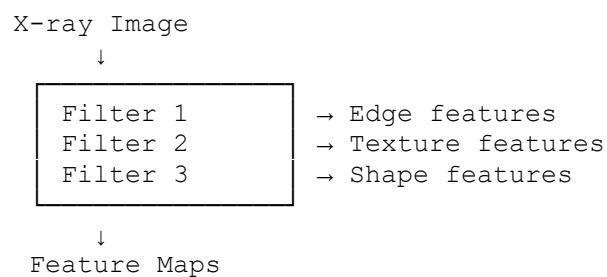
- Horizontal/vertical edges
- Curves
- Textures
- Boundaries
- Abnormal regions

The convolution operation produces **feature maps**.

B. Filters/Kernels

A filter is a small matrix, such as 3×3 , that slides across the X-ray image.

Different filters learn different features automatically.



The filter weights are learned during training through **backpropagation**.

C. Pooling Layers

Pooling reduces the spatial size of feature maps while retaining important information.

Max pooling selects the maximum value from a small region.

For example:

2×2 region → Maximum value → One value

Benefits:

- Reduces computational cost
- Reduces the number of parameters in later layers
- Provides some translation invariance
- Helps retain strong features

D. Fully Connected Layers

After convolution and pooling, the extracted features are passed to the final classification layers.

The fully connected layer combines the learned features to make the final decision:

```
Learned Features
      ↓
Fully Connected Layer
      ↓
Classification
      ↓
Normal / Disease
```

For two classes, a **sigmoid output** can be used. For multiple disease categories, **softmax** is appropriate.

3. Parameter Sharing

One of the major advantages of CNNs is **parameter sharing**.

In a fully connected network, every neuron can be connected to every pixel.

Suppose an image contains:

```
[
224 \times 224 = 50,176
]
```

pixels.

A fully connected neuron would therefore require approximately **50,176 weights**, excluding the bias.

If there are 1,000 neurons:

[
50,176 $\times$ 1,000 = 50,176,000
]

weights are required just for that layer.

CNN approach

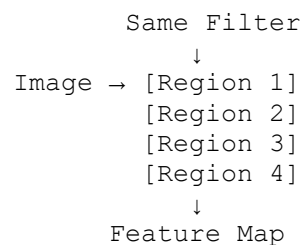
A CNN might use a 3×3 filter.

Therefore, one filter needs only:

[
3 $\times$ 3 = 9
]

weights, plus a bias.

The **same filter is reused across the entire image**.



Therefore, instead of learning separate parameters for every image location, CNNs learn a small set of filters and **share those parameters across spatial locations**.

Result

Parameter sharing:

- Dramatically reduces trainable parameters
- Reduces memory requirements
- Reduces computational cost
- Helps CNNs learn location-independent features such as edges and textures

4. Regularization Techniques

Medical image datasets can still cause **overfitting**, particularly when the model is deep or the number of disease-positive samples is limited.

A. Dropout

Dropout randomly disables a percentage of neurons during training.

Before Dropout:

● ● ● ● ● ●

After Dropout:

● × ● ● × ●

This prevents the network from relying too heavily on particular neurons and improves generalization.

For example, dropout can be applied before the final classification layer.

B. Data Augmentation

Data augmentation artificially creates variations of existing training images.

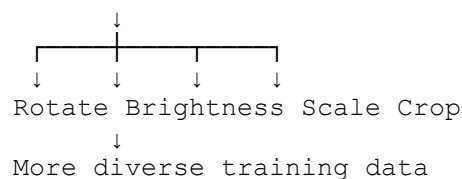
Suitable transformations include:

- Small rotations
- Horizontal flipping where medically appropriate
- Small translations
- Scaling
- Controlled brightness/contrast changes
- Cropping

For X-rays, augmentation must be **medically appropriate**. Excessive transformations could create unrealistic anatomy or remove clinically important information.

Example:

Original X-ray



This helps the model become less sensitive to differences in image acquisition conditions.

C. Batch Normalization

Batch normalization normalizes activations within a mini-batch.

It helps:

- Stabilize training
- Allow faster convergence
- Improve gradient flow
- Provide some regularization
- Make deeper networks easier to train

A typical sequence is:

Convolution → Batch Normalization → ReLU

Using **data augmentation** + **batch normalization** + **dropout** provides stronger protection against overfitting than relying on only one technique.

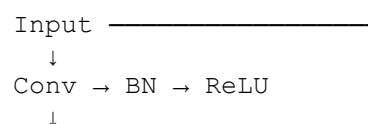
5. AlexNet vs ResNet

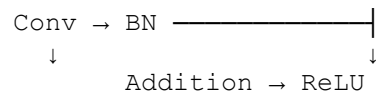
Both **AlexNet** and **ResNet** can perform image classification, but **ResNet would generally be the better choice for this medical application.**

Factor	AlexNet	ResNet
Network depth	Relatively shallow	Much deeper
Feature learning	Good	More powerful hierarchical feature learning
Architecture	Conventional CNN	Uses residual/skip connections
Training deep networks	More difficult	Easier
Vanishing/degradation problems	More likely	Reduced using skip connections
Computational requirement	Lower	Higher
Classification performance	Good baseline	Generally stronger
Transfer learning	Available	Very widely used

Why ResNet?

ResNet introduces **residual/skip connections**:



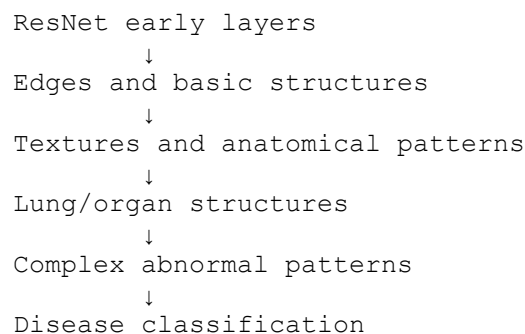


Instead of forcing each layer to learn an entirely new transformation, the network learns a **residual function**.

This makes it possible to train much deeper networks effectively.

Feature Learning

For chest X-rays, deeper networks can learn increasingly complex representations:



Computational Consideration

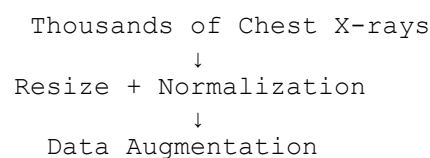
The disadvantage is that deeper ResNet models require **more computation and memory** than AlexNet.

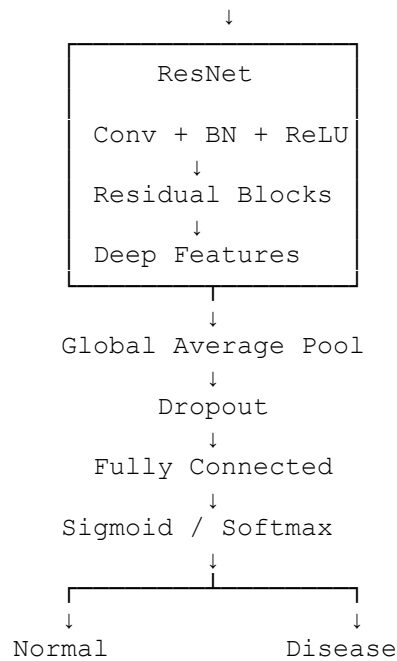
However, this can be addressed using:

- Transfer learning
- GPU acceleration
- Smaller ResNet variants such as ResNet-18 or ResNet-34
- Fine-tuning only selected layers initially

For a healthcare organization, **classification performance and reliable feature learning are generally more important than using the simplest architecture**, provided the computational resources are available.

Final Proposed Solution





Conclusion

A **ResNet-based CNN with convolutional layers, ReLU, batch normalization, pooling/global average pooling, dropout and data augmentation** would be a strong solution for automated chest X-ray classification. CNNs automatically learn hierarchical visual representations, while **parameter sharing greatly reduces the number of trainable parameters** compared with fully connected networks. Among AlexNet and ResNet, **ResNet is preferable** because its deeper architecture and residual connections enable better feature learning and generally stronger classification performance, although it requires greater computational resources.

Case Study Question 2: CNN-Based Autonomous Vehicle Object Recognition (10 Marks) An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.

4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle needs to identify **cars, pedestrians, bicycles, traffic signs, and road obstacles** from camera images. Since the images can vary in lighting, distance, angle, and environment, a deep CNN with strong feature-learning and generalization capability is required.

1. CNN Architecture for Autonomous Vehicle Application

A suitable CNN pipeline can be designed as:

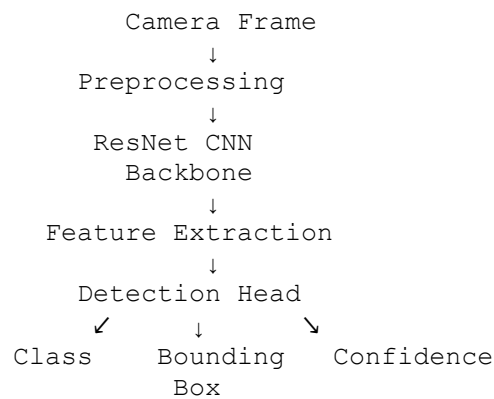
```
Camera Image
↓
Resize + Normalization
↓
Convolution + ReLU
↓
Pooling
↓
Convolution + ReLU
↓
Pooling
↓
Deeper Convolutional Layers
↓
High-Level Features
↓
Fully Connected / Detection Head
↓
Object Classification
↓
Car / Pedestrian / Bicycle / Sign / Obstacle
```

For a real autonomous vehicle, the system should ideally use an **object-detection architecture**, rather than only image classification, because the vehicle needs to know **what the object is and where it is located**.

A practical design can therefore use a CNN backbone such as **ResNet** to extract features, followed by a detection head that produces:

- Object class
- Bounding-box coordinates
- Confidence score

Example:



2. Role of CNN Layers and Progressive Feature Learning

A. Input Layer

The input layer receives images captured by the vehicle's cameras.

For example:

[
Input = $224 \times 224 \times 3$
]

where 3 represents the RGB channels.

Preprocessing can include:

- Resizing
 - Pixel normalization
 - Appropriate image enhancement
 - Data augmentation during training
-

B. Convolutional Layer

The convolutional layer applies small filters/kernels to the image.

For example, a 3×3 **filter** moves across the image and detects local patterns.

The resulting feature maps indicate where particular features occur.

C. Activation Function

After convolution, an activation function such as **ReLU** is applied:

$$\begin{bmatrix} \text{ReLU}(x) = \max(0, x) \end{bmatrix}$$

It introduces non-linearity, allowing the network to learn complex relationships.

D. Pooling Layer

Pooling reduces the spatial dimensions of feature maps.

Max pooling selects the largest value from a local region.

Benefits:

- Reduces computation
 - Reduces feature-map size
 - Retains strong features
 - Provides some translation robustness
-

E. Fully Connected / Output Layer

In a simple image classifier, the learned features are passed to fully connected layers and then to an output layer.

For multiple object classes, the output can provide class probabilities.

However, for an autonomous vehicle, a **detection head** is more appropriate because it must also determine the object's location.

Progressive Feature Learning

One of the major strengths of CNNs is **hierarchical representation learning**.

```
Image
↓
Early CNN Layers
↓
Edges and Corners
↓
Middle Layers
↓
Textures and Shapes
↓
Deep Layers
↓
Object Parts
↓
High-Level Representation
↓
Car / Pedestrian / Bicycle / Sign
```

For example:

- **Early layers:** edges, lines, corners
- **Middle layers:** wheels, windows, traffic-sign shapes, human body parts
- **Deep layers:** complete cars, pedestrians, bicycles, signs, obstacles

The filters are not manually programmed. Their weights are learned during training.

3. Importance of Parameter Sharing

CNNs use the **same filter weights across different spatial locations**.

Suppose a filter learns to detect a vertical edge. The same filter can detect that edge:

```
Left side      Center      Right side
  ↓            ↓            ↓
[Edge]        [Edge]        [Edge]
```

This is called **parameter sharing**.

Advantages

1. Fewer Parameters

A fully connected network would require separate weights for every connection between pixels and neurons.

A CNN instead uses a small filter, such as:

```
[  
3 \times 3 = 9  
]
```

weights per channel, and reuses those weights throughout the image.

2. Lower Computational Cost

Fewer parameters mean:

- Less memory usage
- Fewer calculations
- Faster training
- Faster inference

This is particularly important for autonomous vehicles.

3. Translation-Related Feature Detection

An object may appear at different positions in the image.

For example, a pedestrian could appear:

```
Frame 1: Left side  
Frame 2: Center  
Frame 3: Right side
```

Because the same convolutional filter scans the entire image, the CNN can detect similar features regardless of their exact location.

Thus, parameter sharing helps CNNs recognize **spatially shifted features**.

4. Overfitting and Regularization

An autonomous-vehicle dataset can contain many images, but the model can still overfit to particular roads, cameras, weather conditions, or lighting environments.

For example, the model might perform well on sunny roads but poorly on:

- Night scenes
- Rain
- Fog
- Shadows
- Different camera angles
- Unfamiliar roads

A. Data Augmentation

Generate variations of training images using:

- Rotation
- Scaling
- Cropping
- Translation
- Brightness changes
- Contrast changes
- Blur/noise where realistic
- Weather-related augmentation

This helps the model generalize to different driving environments.

B. Dropout

Dropout randomly disables some neurons during training.

Normal:

• • • • •

Dropout:

• × • • × •

This reduces dependency on individual neurons and helps prevent overfitting.

C. Batch Normalization

Batch normalization normalizes intermediate activations.

It helps:

- Stabilize training
- Improve gradient flow
- Speed up convergence
- Provide some regularization

D. Early Stopping

Training can be stopped when validation performance stops improving.

This prevents the model from continuing to memorize the training data.

E. Transfer Learning

A CNN pretrained on a large image dataset can be fine-tuned using autonomous-driving images.

This is particularly useful when the organization's labeled dataset is limited.

5. AlexNet vs ResNet

Feature	AlexNet	ResNet
Architecture	Conventional CNN	CNN with residual connections
Depth	Relatively shallow	Can be very deep
Feature learning	Good	Strong hierarchical feature learning
Parameter efficiency	Less efficient	Generally more efficient for depth
Training difficulty	Easier than very deep networks	Residual connections make deep training easier
Vanishing gradients	More susceptible	Skip connections greatly reduce the problem
Computational cost	Lower	Higher depending on variant
Representation quality	Good	Generally stronger
Modern applications	Mostly baseline/historical	Widely used as a strong CNN backbone

AlexNet

AlexNet contains several convolutional and fully connected layers and demonstrated that deep CNNs could achieve strong image-recognition performance.

Its advantages include:

- Simpler architecture
- Lower computational requirements
- Faster inference than many deeper networks

However, it has limited depth compared with modern architectures.

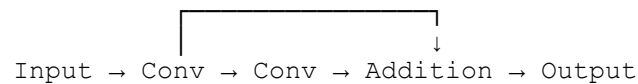
ResNet

ResNet introduces **residual/skip connections**.

Instead of:

Input → Conv → Conv → Output

a residual block provides a shortcut:



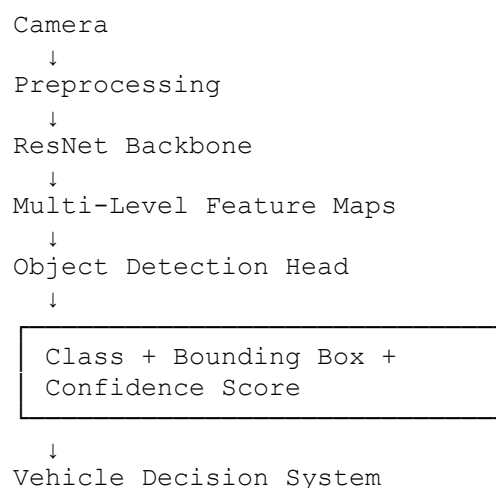
This helps gradients flow through very deep networks and reduces the **vanishing-gradient/degradation problem**.

It allows the network to learn much deeper and more powerful representations.

6. Recommended Architecture

ResNet is the better choice for this autonomous vehicle application.

The primary reason is its stronger ability to learn complex visual representations.



Justification

Accuracy:

ResNet's deeper architecture can learn complex features required to distinguish similar objects such as pedestrians, bicycles, signs, and obstacles.

Feature learning:

Residual networks can learn rich hierarchical representations from low-level edges to high-level object structures.

Training:

Skip connections make deep networks easier to train and reduce problems associated with vanishing gradients.

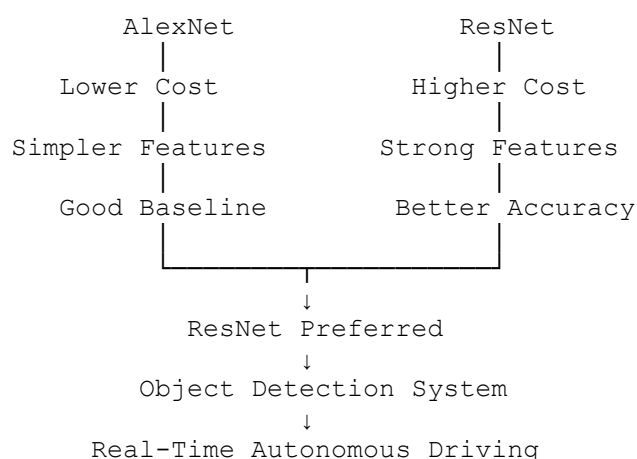
Computational cost:

ResNet is more computationally expensive than AlexNet, but an appropriately sized model such as **ResNet-18** or **ResNet-34** can provide a useful balance between accuracy and efficiency.

Real-time requirements:

Autonomous vehicles require fast inference. Therefore, instead of simply choosing the deepest ResNet available, the system should select a suitable ResNet variant and optimize it using techniques such as:

- GPU/AI accelerator inference
- Model quantization
- Pruning where appropriate
- Efficient input resolution
- Batch size of 1 for low-latency inference

Final Comparison**Conclusion**

For an autonomous vehicle, **ResNet is generally more suitable than AlexNet** because it provides deeper and more powerful feature learning while residual connections make deep

training practical. Although ResNet requires more computational resources, a lightweight variant such as **ResNet-18/34 combined with an efficient object-detection head** can provide a good balance between **accuracy, computational cost, and real-time performance**.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding ; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand